DESIGN AND ANALYSIS OF ALGORITHMS LAB-2

SORTING TECHNIQUES

NAME:D.CHETAN KUMAR

ROLL NO: CH.SC.U4CSE24209

1.Bubble Sort

Code:

```
#include <stdio.h>

int main() {
    int n, i, j, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter elements:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }


    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
```

```
            }
        }
    }

    printf("Sorted array:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Output:

```
chetan@LAPTOP-RO95NMPB:~$ nano bubbleSort.c
chetan@LAPTOP-RO95NMPB:~$ gcc bubbleSort.c -o bubbleSort
chetan@LAPTOP-RO95NMPB:~$ ./bubbleSort
Enter number of elements: 6
Enter elements:
2
9
8
4
7
3
Sorted array:
2 3 4 7 8 9 chetan@LAPTOP-RO95NMPB:~$
```

2.Insertion Sort

Code

```
#include <stdio.h>
int main() {
    int n, i, j, key;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter the elements:\n");
    for(i = 0; i < n; i++) {
```

```
scanf("%d", &arr[i]);
    }

    for(i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while(j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
    printf("Sorted array:\n");
    for(i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Output

```
chetan@LAPTOP-RO95NMPB:~$ nano insertionSort.c
chetan@LAPTOP-RO95NMPB:~$ gcc insertionSort.c -o insertionSort
chetan@LAPTOP-RO95NMPB:~$ ./insertionSort
Enter number of elements:
5
Enter the elements:
2
4
1
9
6
Sorted array:
1 2 4 6 9 chetan@LAPTOP-RO95NMPB:~$
```

3.Selection Sort

Code

```
#include <stdio.h>

int main() {
    int n, i, j, minIndex, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter the elements:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    for(i = 0; i < n - 1; i++) {
        minIndex = i;

        for(j = i + 1; j < n; j++) {
            if(arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }

        temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
```

```
    printf("Sorted array:\n");
    for(i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Output:

```
chetan@LAPTOP-RO95NMPB:~$ nano selectionSort.c
chetan@LAPTOP-RO95NMPB:~$ gcc selectionSort.c -o selectionSort
chetan@LAPTOP-RO95NMPB:~$ ./selectionSort
Enter number of elements: 5
Enter the elements:
3
9
7
1
8
Sorted array:
1 3 7 8 9 chetan@LAPTOP-RO95NMPB:~$
```

4.Heap Sort

Code:

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2*i + 1;
```

```
    int right = 2*i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}


void heapSort(int arr[], int n) {

    for (int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);


    for (int i = n - 1; i > 0; i--) {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);

    int arr[n];
    printf("Enter elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    heapSort(arr, n);

    printf("Sorted array:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

Output:

```
chetan@LAPTOP-RO95NMPB:~$ nano heapSort.c
chetan@LAPTOP-RO95NMPB:~$ gcc heapSort.c -o heapSort
chetan@LAPTOP-RO95NMPB:~$ ./heapSort
Enter number of elements: 6
Enter elements:
7
1
6
2
4
9
Sorted array:
1 2 4 6 7 9 chetan@LAPTOP-RO95NMPB:~$
```

5.Bucket Sort

Code:

```
#include <stdio.h>
void insertionSort(float arr[], int n) {
    for(int i = 1; i < n; i++) {
        float key = arr[i];
        int j = i - 1;
        while(j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    float arr[n];
    printf("Enter elements (0 to 1):\n");
    for(int i = 0; i < n; i++)
        scanf("%f", &arr[i]);
    float bucket[n][n];
    int count[n];
    for(int i = 0; i < n; i++)
        count[i] = 0;
    for(int i = 0; i < n; i++) {
        int index = n * arr[i];
        bucket[index][count[index]++] = arr[i];
    }
    for(int i = 0; i < n; i++)
```

```
        insertionSort(bucket[i], count[i]);
    int k = 0;
    for(int i = 0; i < n; i++)
        for(int j = 0; j < count[i]; j++)
            arr[k++] = bucket[i][j];

    printf("Sorted array:\n");
    for(int i = 0; i < n; i++)
        printf("%.2f ", arr[i]);

    return 0;
}
```

Output:

```
chetan@LAPTOP-RO95NMPB:~$ nano bucketSort.c
chetan@LAPTOP-RO95NMPB:~$ gcc bucketSort.c -o bucketSort
chetan@LAPTOP-RO95NMPB:~$ ./bucketSort
Enter number of elements: 6
Enter elements (0 to 1):
0.42
0.32
0.23
0.52
0.25
0.21
Sorted array:
0.21 0.23 0.25 0.32 0.42 0.52 chetan@LAPTOP-RO95NMPB:~$
```